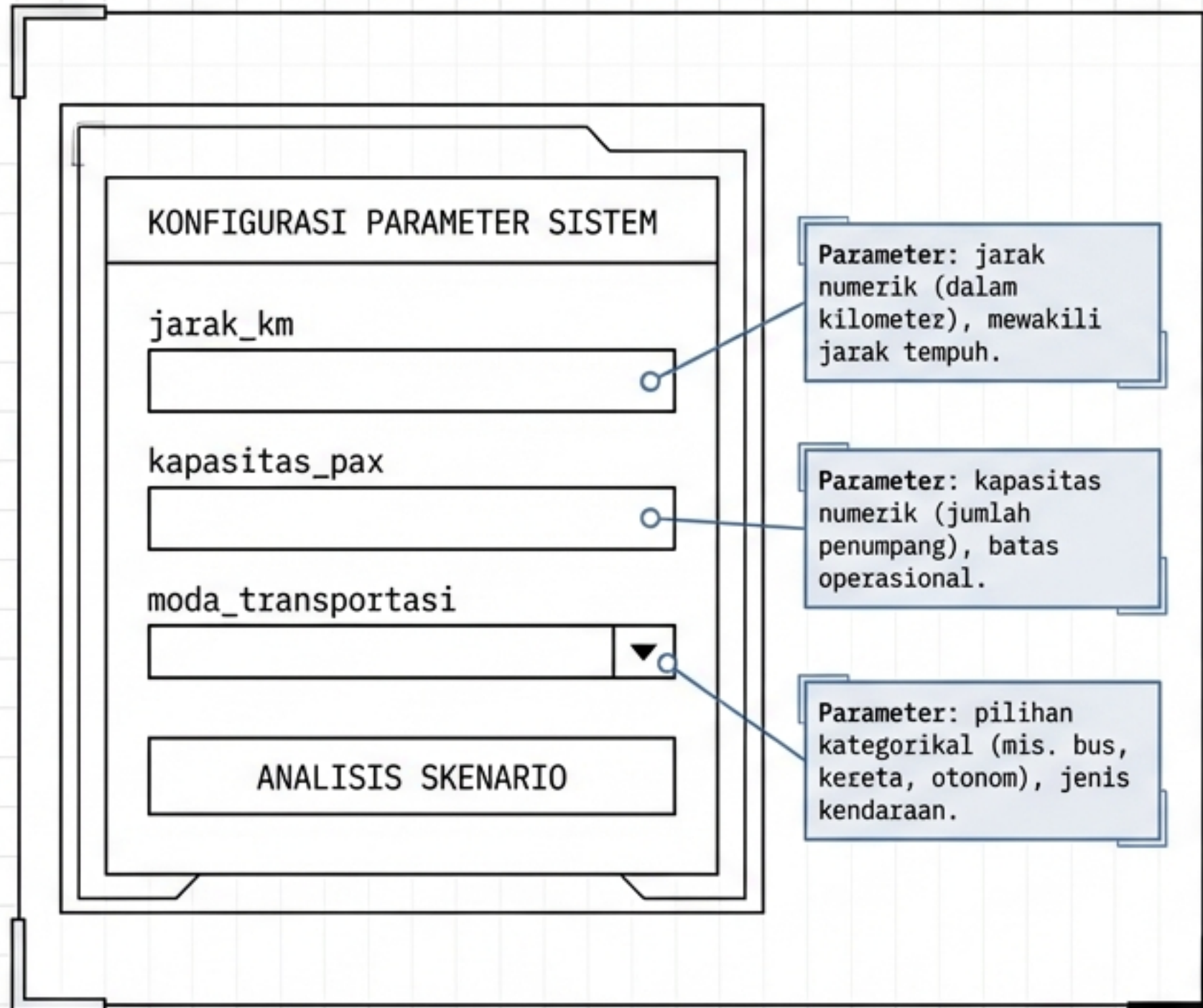


Gambar 31: Antarmuka Pengguna

Titik Masuk Manusia ke Dalam Sistem Cerdas



Pengumpulan Parameter:

Antarmuka bertindak sebagai titik awal siklus persepsi sistem cerdas.

Penerjemahan Niat:

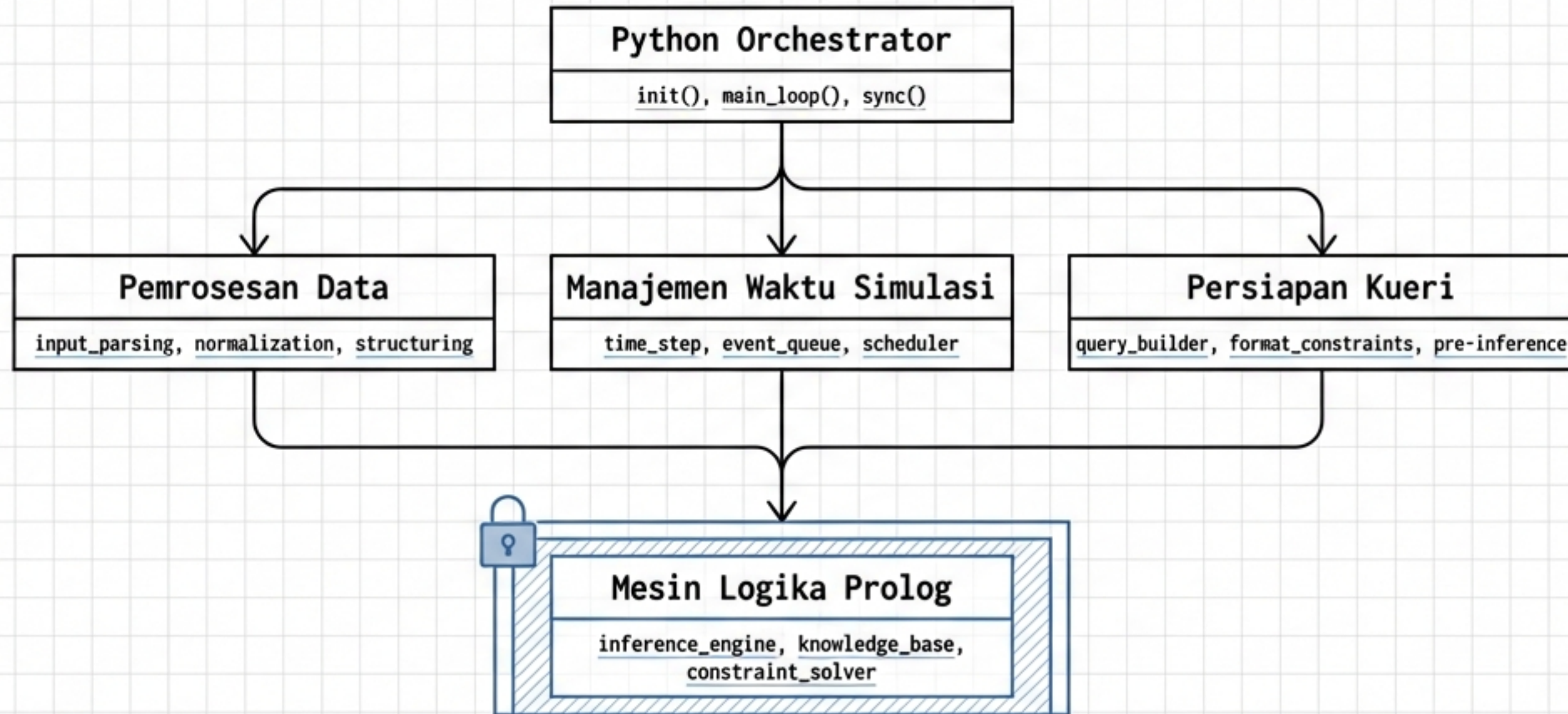
Mengubah input manusia (skenario masa depan, kendala operasional) menjadi struktur data terstandarisasi.

Batas Sederhana:

Menjaga kompleksitas logika sistem tetap tersembunyi, memberikan kontrol penuh tanpa membebani pengguna akhir.

Gambar 32: Orkestrasi Simulasi

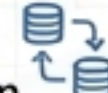
Python Sebagai Konduktor Orkestrasi Multi-Modul



Manajemen Alur Kerja:
Mengontrol inisialisasi, perulangan simulasi, dan sinkronisasi waktu.



Transformasi Data:
Memproses data input numerik dan terstruktur sebelum dikirim ke mesin penalaran logika.

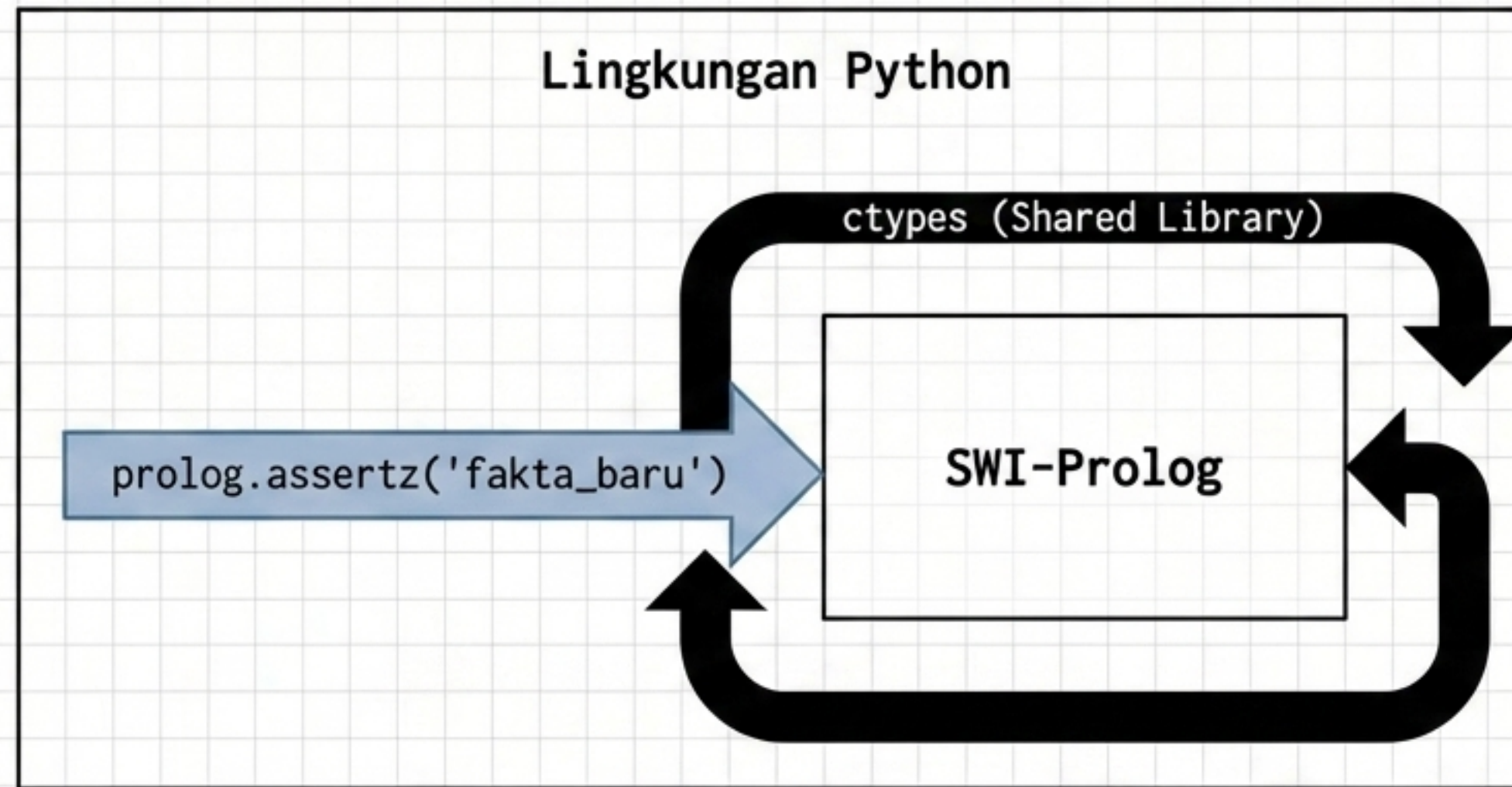


Pemisahan Tugas:
Python menangani algoritma komputasional dan antarmuka, membebaskan lapisan logika untuk fokus murni pada inferensi dan konstrain.



Gambar 33: PySWIP

Injeksi Fakta Dinamis via Shared Library



Mekanisme Inti:

Beroperasi dengan memanggil SWI-Prolog sebagai shared library menggunakan antarmuka ctypes Python.

Pembaruan Status Dinamis:

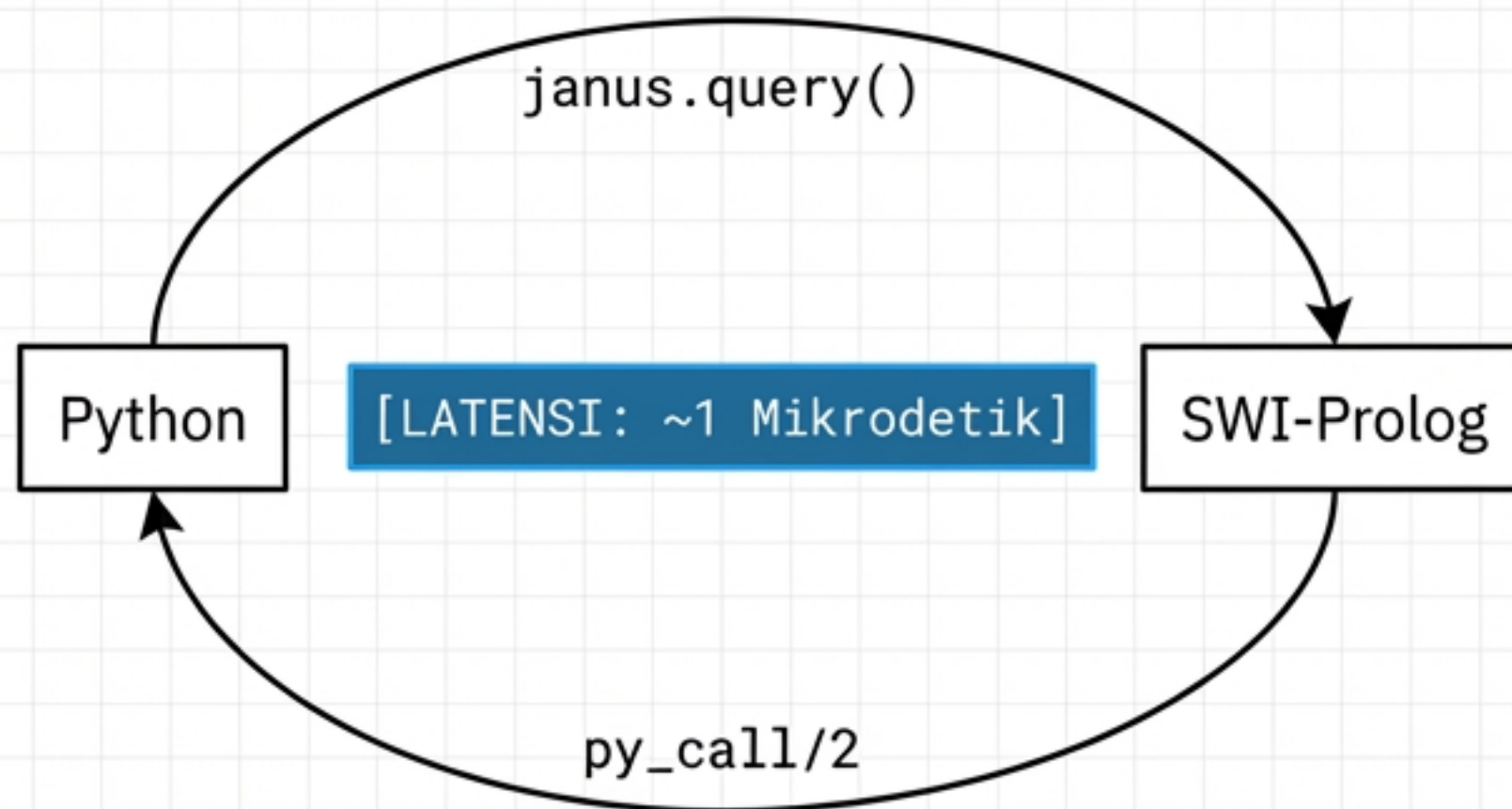
Sangat ideal untuk simulasi yang membutuhkan injeksi fakta baru secara real-time (contoh: mengubah status operasional dalam skenario berjalan).

Eksekusi Kueri:

Mengirimkan kueri dari Python dan menerima solusi sebagai dictionary Python yang siap diproses ulang.

Gambar 34: Janus

Komunikasi Dua Arah Berkinerja Tinggi



Integrasi C-API:

Dibangun langsung di atas API C tingkat rendah untuk mencapai komunikasi dengan latensi nyaris nol (~1 mikrodetik).

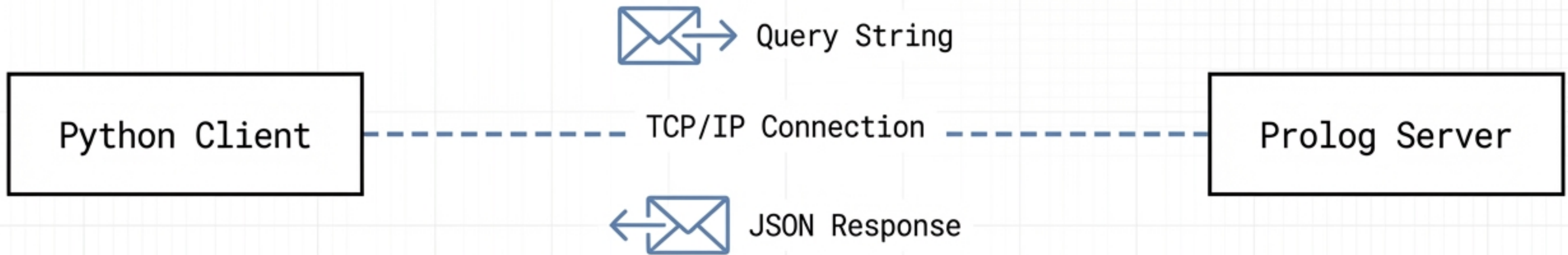
Pertukaran Data Dua Arah:

Memungkinkan Python memanggil predikat Prolog secara fungsional, dan sebaliknya.

Konversi Tipe Transparan:

Pemetaan otomatis yang mulus antara dictionary Python dan dict SWI-Prolog.

Gambar 35: PrologMQI Arsitektur Jaringan Antrian Pesan



Pemisahan Klien-Server:

- Menggunakan pendekatan Message Queue Interface berbasis jaringan, bukan integrasi pustaka internal.

Format Universal:

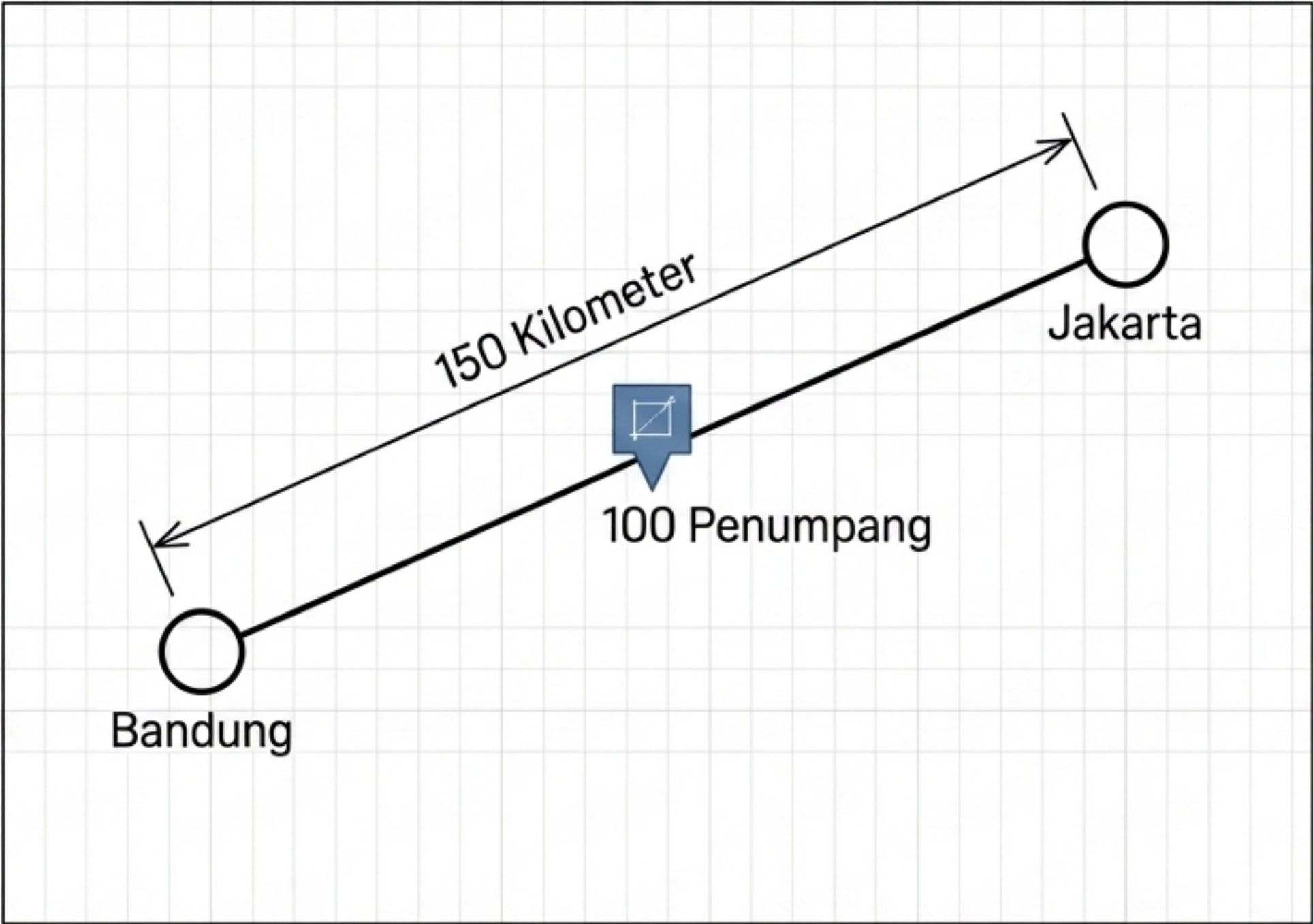
- Permintaan dikirim sebagai string murni; hasil dikembalikan dalam format JSON standar.

Skalabilitas Terdistribusi:

- Sangat cocok untuk sistem arsitektur mikro (microservices) atau integrasi di mana Python dan Prolog berada pada server fisik yang berbeda.

Gambar 36: Skenario Bandung-Jakarta

Parameter Simulasi Transportasi 2030



Tujuan Eksperimen:
Mengevaluasi kinerja sistem cerdas dalam menyelesaikan masalah multi-domain (transportasi, lingkungan, ekonomi).
Konstrain Koridor:
- Jarak Tempuh: 150 Kilometer - Permintaan (Load): Memindahkan 100 Penumpang - Konteks Tahun: Proyeksi infrastruktur dan grid energi 2030
Pusat Kendali:
Seluruh parameter ini dimasukkan via Antarmuka dan dieksekusi oleh Orkestrator yang terhubung ke Ontologi Prolog.

Gambar 37: Subjek Uji

Tiga Alternatif Moda Transportasi

Kendaraan Listrik Pribadi (EV)



Fleksibel, namun menuntut infrastruktur masif. Beroperasi pada asumsi grid bauran energi PLN 2030.

Kebutuhan: 25 Unit (untuk 100 pax)

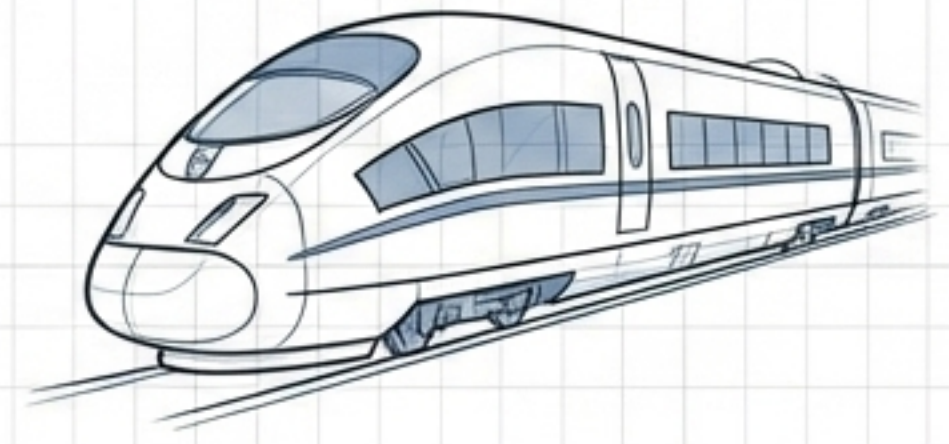
Bus Diesel Euro6



Solusi transit massal konvensional. Mengandalkan efisiensi bio-diesel (B30) dengan kapasitas sedang.

Kebutuhan: 3 Unit (untuk 100 pax)

Kereta Cepat Listrik (HST)

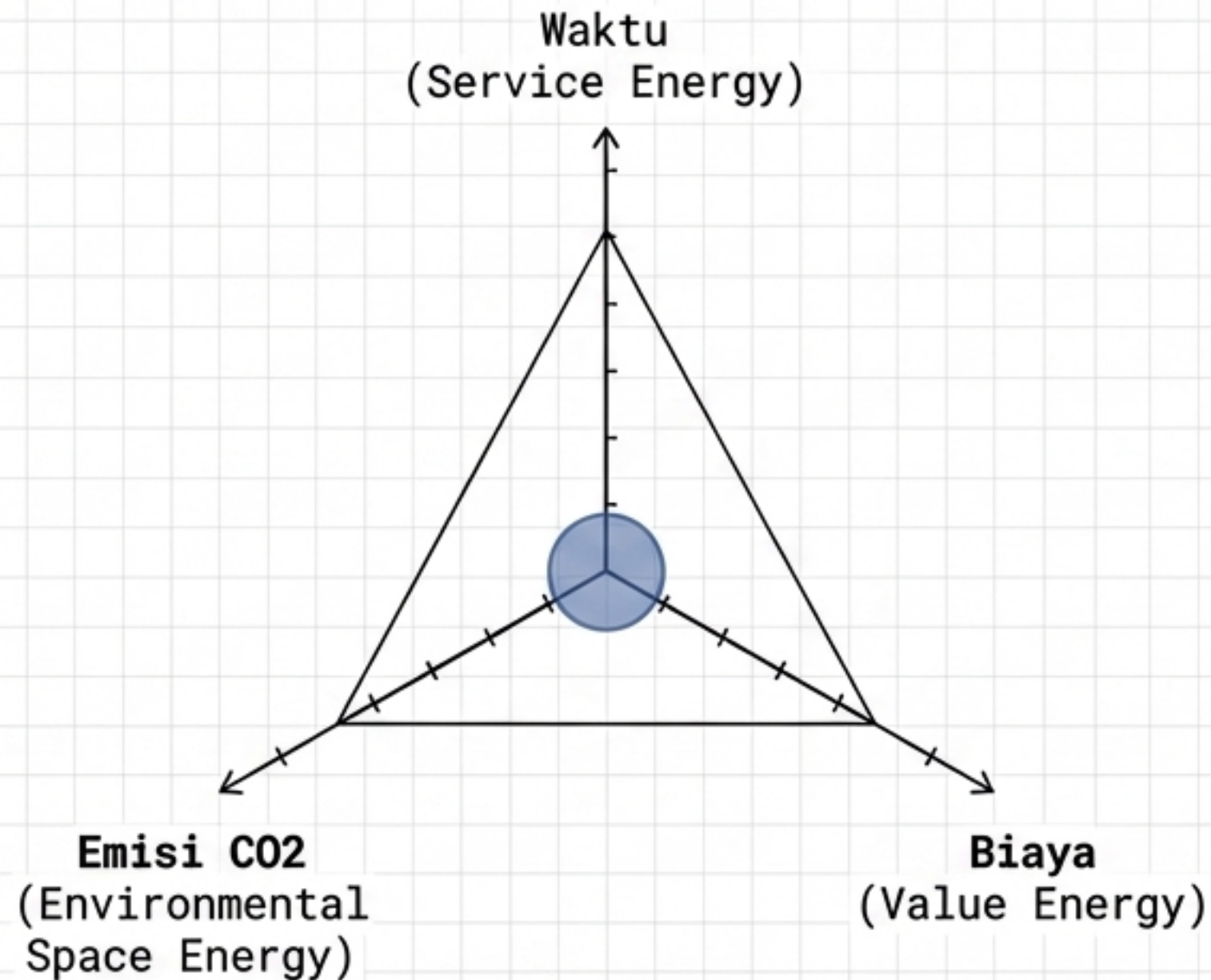


Solusi jalur tetap, berkecepatan tinggi, dengan investasi awal raksasa dan emisi karbon rendah.

Kebutuhan: 1 Rangkaian

Gambar 38: Metrik & Hasil Evaluasi

Sumbu Evaluasi PSKVE dan Trade-Off Multi-Dimensi



Hasil Simulasi (Per Penumpang):

Kereta Cepat (HST):

- Waktu: 1.0 - 1.5 jam (Tercepat)
- Biaya: 150,000 IDR (Tertinggi)
- Emisi: ~840g CO2 (Terendah)

Bus Diesel Euro6:

- Waktu: 3.0 - 4.0 jam (Terlama)
- Biaya: ~76,381 IDR (Terendah)
- Emisi: ~3,234g CO2 (Tertinggi)

Armada EV Sedan:

- Waktu: 2.5 - 3.5 jam (Menengah)
- Biaya: ~107,750 IDR (Menengah)
- Emisi: ~1,575g CO2 (Moderasi Grid)

Kesimpulan:

Ontologi secara otomatis mengevaluasi transaksi energi untuk memfasilitasi keputusan holistik yang melampaui metrik tunggal.